

CS50's Introduction to Programming with R

OpenCourseWare

Donate  (<https://cs50.harvard.edu/donate>)

Carter Zenke (<https://carterzenke.me>)

carter@cs50.harvard.edu

 (<https://github.com/carterzenke>)  (<https://www.linkedin.com/in/carterzenke/>)

David J. Malan (<https://cs.harvard.edu/malan/>)

malan@harvard.edu

 (<https://www.facebook.com/dmalan>)  (<https://github.com/dmalan>) 

(<https://www.instagram.com/davidjmalan/>)  (<https://www.linkedin.com/in/malan/>) 

(<https://www.reddit.com/user/davidjmalan>)  (<https://www.threads.net/@davidjmalan>) 

(<https://twitter.com/davidjmalan>)

Lecture 7

- [Welcome!](#)
- [Packages](#)
- [Package Structure](#)
- [devtools](#)
- [Writing Tests](#)
- [Writing R Code](#)
- [NAMESPACE](#)
- [Testing Code](#)
- [Writing Documentation](#)
- [Building Packages](#)
- [Updating Packages](#)
- [Using and Sharing Packages](#)
- [Summing Up](#)

Welcome!

- Welcome back to CS50's Introduction to Programming with R!
- Today, we will be learning about packaging and distributing our programs. This way, we can share them with the world!

Packages

- Today, we are going to be building and packaging a program called `ducksay`.
- If you have taken one of our other CS50 courses, you might be aware of a program called `cowsay`. It takes some piece of text and creates an image of a cow saying that text. We will be building `ducksay` in that same spirit.
- *Packages* are source code that has been compiled so that it can be distributed.

Package Structure

- Packages should be held within a folder that is the same name as the package.
- We can do so by typing the following commands in the R console:

```
dir.create("ducksay")  
setwd("ducksay")
```

Notice that these commands create a directory called `ducksay` and then set the working directory to `ducksay`.

- Packages generally have a structure as follows within the main folder:

```
DESCRIPTION  
NAMESPACE  
man/  
R/  
tests/
```

The `DESCRIPTION` file will include a description of the package, including who wrote it. The `NAMESPACE` file will include a list of functions we want to make available to the users of our package. `man` is a folder that holds the manual (documentation) for the package. `R` includes the R code for the package. Finally, `tests` holds all the tests that we want to be able to run to ensure our package behaves as we expect.

- We can create a `DESCRIPTION` file by typing `file.create("DESCRIPTION")` into the R console. We can now open this file and code as follows:

```
# Demonstrates required components of a DESCRIPTION file  
  
Package: ducksay  
Title: Duck Say  
Description: Say hello with a duck.  
Version: 1.0  
Authors@R: person("Carter", "Zenke", email = "carter@cs50.harvard.edu", role =  
License: MIT + file LICENSE
```

Notice how the package is named and titled. Then, a description is provided. Authors are included. Finally, the license is provided under which this package is offered. You can learn more about these fields in the `DESCRIPTION` file [documentation \(https://cran.r-project.org/doc/manuals/R-exts.html#The-DESCRIPTION-file\)](https://cran.r-project.org/doc/manuals/R-exts.html#The-DESCRIPTION-file).

- As you can see by the `DESCRIPTION` file above, we also need a `LICENSE` file. We can code that as follows:

```
# Demonstrates adding on to a license template

YEAR: ...
COPYRIGHT HOLDER: ducksay authors
```

Fill in the `...` with the present year. Notice how the year of the license and the copyright holder is named.

devtools

- A package called devtools allows us to create packages faster.
- In particular, `devtools` comes with utilities for creating the necessary folder structure for our package's tests and R code.
- We can load devtools by typing `library(devtools)` into the R console, assuming it's already installed.

Writing Tests

- Thanks to the devtools package, we can easily use testthat to develop tests for packages we author.
- Then, we can type `use_testthat()` to invoke the ability to use testthat. Our `DESCRIPTION` file will be automatically modified as follows:

```
# Demonstrates suggesting a dependency, for testing's sake

Package: ducksay
Title: Duck Say
Description: Say hello with a duck.
Version: 1.0
Authors@R: person("Carter", "Zenke", email = "carter@cs50.harvard.edu", role =
License: MIT + file LICENSE
Suggests:
  testthat (>= 3.0.0)
Config/testthat/edition: 3
```

Notice that the package will suggest that one should have testthat version 3.0.0 or above installed. This may vary depending on the version of testthat that you've installed.

- Inside our `tests/testthat` folder, created by `use_testthat`, we can create our first test, `test-ducksay.R`, as follows:

```
# Demonstrates describing behavior of `ducksay`

describe("ducksay()", {
  it("can print to the console with `cat`", {
```

```
expect_output(cat(ducksay()))
})
it("can say hello to the world", {
  expect_match(ducksay(), "hello, world")
})
})
```

Notice that `expect_match` looks for the string `hello, world` in the output of `ducksay`.

Writing R Code

- Continuing our use of devtools we can now type the following in the R console:
`use_r("ducksay")`.
- This command will create a folder called `R` and a file called `ducksay.R`.
- Now, it's time to provide some functionality within our program that we will package. This functionality should match the test that we wrote.
- Write code for `ducksay.R` as follows:

```
# Demonstrates defining a function for a package

ducksay <- function() {
  paste(
    "hello, world",
    ">(.)_",
    " (___)/",
    sep = "\n"
  )
}
```

NAMESPACE

- As mentioned previously, we need to provide information about what functions are available to the end users of this package in a file called `NAMESPACE`.
- To do so, you can type `file.create("NAMESPACE")` in your console. Then, edit this file as follows:

```
# Demonstrates declaring `ducksay` accessible to package end users

export(ducksay)
```

This file simply makes the `ducksay` function available to the end user of the package.

- We can now type `load_all()` into the R console to load all the available functions named in `NAMESPACE`.

Testing Code

- You can now run `test()` to test our function. No errors should result.
- Further, you can now use `ducksay` in RStudio after you have loaded the functions of this package.
- Updating our tests, let's test to see if a duck appears in `ducksay`:

```
# Demonstrates checking for duck in output

describe("ducksay()", {
  it("can print to the console with `cat`", {
    expect_output(cat(ducksay()))
  })
  it("can say hello to the world", {
    expect_match(ducksay(), "hello, world")
  })
  it("can say hello with a duck", {
    duck <- paste(
      ">( . )__",
      " (____)/",
      sep = "\n"
    )
    expect_match(ducksay(), duck, fixed = TRUE)
  })
})
```

Notice how this test looks to see if a duck is represented. Additionally, notice how `fixed = TRUE`, as described in the lecture, prevents the test from misinterpreting some of the characters within the duck as being part of something called a regular expression. Suffice to say for now, a regular expression is not what we'd like!

Writing Documentation

- We can now document how to use our function. Typically, we can type `?ducksay` to see documentation. However, we have not created our documentation yet.
- Documentation is written in a type of language called a markup language. A markup language provides syntax for specifying the formatting of a document.
- You can write your documentation by typing:

```
dir.create("man")
file.create("man/ducksay.Rd")
```

The first command creates a folder called `man`. The second creates our documentation file.

- Modify your documentation file as follows:

```
# Demonstrates required markup for R documentation files

\name{ducksay}
\alias{ducksay}
\title{Duck Say}
\description{A duck that says hello.}
```

```

\usage{
ducksay()
}
\value{
A string representation of a duck saying hello to the world.
}
\examples{
cat(ducksay())
}

```

Notice how the `name`, `title`, `description`, `usage`, and other sections are provided. You can learn more about these elements by reading the [documentation \(https://cran.r-project.org/doc/manuals/R-exts.html#Writing-R-documentation-files\)](https://cran.r-project.org/doc/manuals/R-exts.html#Writing-R-documentation-files) on R documentation files.

- Now, it's time to wrap things up and share our package.

Building Packages

- Once a package's contents are ready to be bundled together and distributed, one of two commands can be used to initiate a *build*:

```

build
R CMD build

```

Notice that `build` is a devtools function and can be run directly from the R console. `R CMD build` can be run from the computer's terminal outside of R.

- Running `build`, you will see a `.gz` file will be outputted within your working directory.

Updating Packages

- To update our code, we can open our test file and update our test as follows:

```

# Demonstrates ensuring duck repeats given phrase

describe("ducksay()", {
  it("can print to the console with `cat`", {
    expect_output(cat(ducksay()))
  })
  it("can say hello to the world", {
    expect_match(ducksay(), "hello, world")
  })
  it("can say hello with a duck", {
    duck <- paste(
      ">(.)__",
      " (___)/",
      sep = "\n"
    )
    expect_match(ducksay(), duck, fixed = TRUE)
  })
  it("can say any given phrase", {
    expect_match(ducksay("quack!"), "quack!")
  })
})

```

```
    })
  })
```

Notice that a new test is added that looks for “quack!”

- With this test in mind, we can now update our source code to allow for the inputting of any phrase, which, in turn, will be stated by the duck:

```
# Demonstrates taking an argument to print

ducksay <- function(phrase = "hello, world") {
  paste(
    phrase,
    ">( . ) _",
    " ( ____ ) /",
    sep = "\n"
  )
}
```

Notice that a default `phrase`, “hello, world” is provided. If another `phrase` is provided, it will say that `phrase` instead.

- Similarly, we can update our documentation file as follows:

```
# Demonstrates updated markup, including specifying arguments

\name{ducksay}
\alias{ducksay}
\title{Duck Say}
\description{A duck that says hello.}
\usage{
ducksay(phrase = "hello, world")
}
\arguments{
\item{phrase}{The phrase for the duck to say.}
}
\value{
A string representation of a duck saying the given phrase.
}
\examples{
cat(ducksay())
cat(ducksay("quack!"))
}
```

Notice that `value` is updated. Further, `arguments` is also updated. Another example is provided in `examples`.

- We can run `build` again to include our modifications.
- This package can now be shared with others.

Using and Sharing Packages

- Let's now create a program called `greet.R` that uses this package.

- We can set the working directory away from `ducksay` by typing `setwd("../")` in the R console. This will move our working directory to one directory higher than that of `ducksay`.
- Next, we can type `file.create("greet.R")` to create a new file. Modify this file as follows:

```
# Demonstrates using custom package

library(ducksay)

name <- readline("What's your name? ")
greeting <- ducksay(paste("hello,", name))
cat(greeting)
```

Notice that this program loads `ducksay`. Then, the code uses this new library.

- While this works on our computer, being that we developed this package locally, others will need to install this package. To do so, one of the following commands can be used:

```
install.packages
R CMD INSTALL
```

As discussed in previous lectures, the top command can be run directly within RStudio and is built into R itself. The other command can be run in the computer's terminal. It is also built into R.

- To install our package, we can run the following in our console:
`install.packages("ducksay_1.0.tar.gz")`
- You can share your code using CRAN, GitHub, and even email.

Summing Up

In this lesson, you learned how to package your programs in R. Specifically, you learned about:

- Packages
- Package Structure
- devtools
- Writing Tests
- Writing R Code
- `NAMESPACE`
- Testing Code
- Writing Documentation
- Building Packages
- Updating Packages
- Using and Sharing Packages

In this course, you learned many things about R and programming with R. You learned how to represent data, transform data, apply functions, tidy data, visualize data, test programs, and package programs. In all, we hope that you have found this material helpful to you. We also hope that you will take what you have learned to do great good in the world.

This was CS50's Introduction to Programming with R.